COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	16/07/2026	**DATE SUBMITTED:**	16/07/2026
NAME:	Ramon Brendt D. Campo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 2.1 [Arrays, Pointers and Dynamic Memory Allocation]

1. PROCEDURE OUTPUT

Table 2-1. Initial Driver Program

SCREENSHOTS

```
//libraries
#include <iostream>
#include <string.h>  //to use and store strings in the program

//creates a class called student
class student{
private:          //variables that can be only accessed within the class
std:: string studentName;
int studentAge ;

public:
//constructor

student(std:: string newname = "john doe", int newage = 18){     //assigns the "student" class with variables newname and newage
studentName = std:: move(newname); //NOTE! the 'move' command is basically copy assignment opperator but instead of copying its moving it-
                                   //-think of it as append and write in oop2
studentAge = newage;
std:: cout << "the constructor is called" << std:: endl;
}

//deconstructor

//!NOTE the destructor function's name does not REFER to the constructor function "student" instead it referrs to the "student" class
~student(){

    std:: cout << "destructor is called" << std:: endl;


}
```

```cpp
//copy constructor


student(const student &copystudent){

    std:: cout << "copy constructor is called" << std:: endl;

    //copies the EXACT value of studentName and studentAge in the private section and creates a new copy that is also named studentName -
        //-and has the contents of the original student name in it

    //to differentiate the 2 thats we added &copystudent to get the data of the original studentName and assigns it to the new studentName
        //-variable which has a different address but still holds the original value inside of it

    //FAQ: why use & when you can directly assign?
    //it is use to PASS the data to the constructor but instead of duplicating the data it actually creates an alias which in this case is-
        //- the student class itself
    //because ithout '&', passing by value would require calling the copy constructor to set up the argument, triggering infinite recursion
        //-that overflows the stack and crashes.

    studentName = copystudent.studentName;
    studentAge = copystudent.studentAge;

}
```

```cpp
 // Copy Assignment Operator

 //the function is only called if you assign a variable with another variable (ex. student3 = student2)
 // FAQ: why does it use a referrence "&"?: so that the variable can be assigned/allocate multiple times which in the function's case would
        //- be the variable student2
 student& operator=(const student &copystudent) {
    std::cout << "copy assignment operator is called" << std::endl;

    // checks if the assigned variable has the same name as the assigned variable (ex. student3 = student3)
    // the "this" variable is a variable that is used in classes to store dereferrenced data in it and it is promised to return it as-
        //-referrenced data
    if (this == &copystudent) {
        return *this;   //dereferrences &copystudent
    }

    // creates a copy
    studentName = copystudent.studentName;
    studentAge = copystudent.studentAge;

    // returns the dereferrenced &copystudent
    return *this;
 }
```

```
void Print_Details (){

  std::cout << this->studentName << " " << this->studentAge << std::endl;

};
};

//main caler
int main(){


  student student1("andy", 20) ;      //assigns student1 with andy and 20
  student student2(student1);         //creates a copy of student1 named student2
  student student3;
  student3 = student2 ;              //assigns the contents of student2 to student 3

return 0;

}
```

OUTPUT

```
the constructor is called
copy constructor is called
the constructor is called
copy assignment operator is called
destructor is called
destructor is called
destructor is called
PS C:\Users\PC\Documents\DSA CAMPO>
```

OBSERVATION

-from what I've noticed from this code is that the inputs made in the main caller shows how each three functions (destructor, copy constructor, copy assignment) works like for example in student 1 it assigns the data in it which is andy and 20 which will trigger the constructor function moving the data imputed in student1, while in student 2 its assigned the values inside student1 which means it will create a copy of student1 thus triggering the copy constructor function, which creates a an exact copy of the variable and the values in it this being the variable "student2" and lastly student3 it gets assigned by student2 which will trigger the assignment function, that checks the input if it has an = or a variable assigned to it, if it has a = or assignment to it, it will create a copy of the data inside and creates a new variable to place it and that variable is called "student3"

Table 2-2. Modified Driver Program with Student Lists

SCREENSHOTS

```cpp
//libraries
#include <iostream>
#include <string.h>  //to use and store strings in the program

//creates a class called student
class student{
private:          //variables that can be only accessed within the class
std:: string studentName;
int studentAge ;

public:
//constructor

student(std:: string newname = "john doe", int newage = 18){     //assigns the "student" class with variables newname and newage
studentName = std:: move(newname); //NOTE! the 'move' command is basically copy assignment opperator but instead of copying its moving it-
                                   //-think of it as append and write in oop2
studentAge = newage;
std:: cout << "the constructor is called" << std:: endl;
}

//deconstructor

//!NOTE the destructor function's name does not REFER to the constructor function "student" instead it referrs to the "student" class
~student(){

    std:: cout << "destructor is called" << std:: endl;


}
```

```cpp
//copy constructor


student(const student &copystudent){

    std:: cout << "copy constructor is called" << std:: endl;

    //copies the EXACT value of studentName and studentAge in the private section and creates a new copy that is also named studentName -
        //-and has the contents of the original student name in it

    //to differentiate the 2 thats we added &copystudent to get the data of the original studentName and assigns it to the new studentName
        //-variable which has a different address but still holds the original value inside of it

    //FAQ: why use & when you can directly assign?
    //it is use to PASS the data to the constructor but instead of duplicating the data it actually creates an alias which in this case is-
        //- the student class itself
    //because ithout '&', passing by value would require calling the copy constructor to set up the argument, triggering infinite recursion
        //-that overflows the stack and crashes.

    studentName = copystudent.studentName;
    studentAge = copystudent.studentAge;

}
```

```cpp
// Copy Assignment Operator

//the function is only called if you assign a variable with another variable (ex. student3 = student2)
// FAQ: why does it use a referrence "&"?: so that the variable can be assigned/allocate multiple times which in the function's case would
        //- be the variable student2
student& operator=(const student &copystudent) {
    std::cout << "copy assignment operator is called" << std::endl;

    // checks if the assigned variable has the same name as the assigned variable (ex. student3 = student3)
    // the "this" variable is a variable that is used in classes to store dereferrenced data in it and it is promised to return it as-
        //-referrenced data
    if (this == &copystudent) {
        return *this;   //dereferrences &copystudent
    }

    // creates a copy
    studentName = copystudent.studentName;
    studentAge = copystudent.studentAge;

    // returns the dereferrenced &copystudent
    return *this;
}
```

```cpp
 void Print_Details (){

  std::cout << this->studentName << " " << this->studentAge << std::endl;

 };
 };

 //main caler
 int main() {

  const size_t j = 5;                    //size_t represents the size of arrays in bytes. assigns j with 5

  student studentList[j] = {};                                  //asigns the size of the array which will be stored in the "student" class

  //assigns the name and age of the students
  std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
  int ageList[j] = {15, 16, 18, 19, 16};

 return 0;
 }
```

OUTPUT:

```
the constructor is called
the constructor is called
the constructor is called
the constructor is called
the constructor is called
destructor is called
destructor is called
destructor is called
destructor is called
destructor is called
PS C:\Users\PC\Documents\DSA CAMPO>
```

Observations:

I noticed that the copy constructor and the copy assignment operator isn't called which makes sense because the input in the main caller didn't tell or trigger any copy function here. It simply stored the data values (age and name of the students) to the constructor function to construct and once its done its then destroyed by the destructor

Table 2-3. Final Driver Program

SCREENSHOTS:

```
//libraries
#include <iostream>
#include <string.h>  //to use and store strings in the program

//creates a class called student
class student{
private:          //variables that can be only accessed within the class
std:: string studentName;
int studentAge ;

public:
//constructor

student(std:: string newname = "john doe", int newage = 18){     //assigns the "student" class with variables newname and newage
studentName = std:: move(newname); //NOTE! the 'move' command is basically copy assignment opperator but instead of copying its moving it-
                                   //-think of it as append and write in oop2
studentAge = newage;
std:: cout << "the constructor is called" << std:: endl;
}

//deconstructor

//!NOTE the destructor function's name does not REFER to the constructor function "student" instead it referrs to the "student" class
~student(){

    std:: cout << "destructor is called" << std:: endl;


}
```

```
//copy constructor


student(const student &copystudent){

    std:: cout << "copy constructor is called" << std:: endl;

    //copies the EXACT value of studentName and studentAge in the private section and creates a new copy that is also named studentName -
        //-and has the contents of the original student name in it

    //to differentiate the 2 thats we added &copystudent to get the data of the original studentName and assigns it to the new studentName
        //-variable which has a different address but still holds the original value inside of it

    //FAQ: why use & when you can directly assign?
    //it is use to PASS the data to the constructor but instead of duplicating the data it actually creates an alias which in this case is-
        //- the student class itself
    //because ithout '&', passing by value would require calling the copy constructor to set up the argument, triggering infinite recursion
        //-that overflows the stack and crashes.

    studentName = copystudent.studentName;
    studentAge = copystudent.studentAge;

}
```

```
void Print_Details (){

  std::cout << this->studentName << " " << this->studentAge << std::endl;

};
};


//main caler
int main() {

 const size_t j = 5;                    //size_t represents the size of arrays in bytes. assigns j with 5

 student studentList[j] = {};                              //asigns the size of the array which will be stored in the "student" class

 //assigns the name and age of the students
 std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
 int ageList[j] = {15, 16, 18, 19, 16};

 for(int i = 0; i < j; i++){ //loop A
 student *ptr = new student(namesList[i], ageList[i]);
 studentList[i] = *ptr;
 }

 for(int i = 0; i < j; i++){ //loop B
 studentList[i].Print_Details();
 }
 return 0;
}
```

LOOP A:

```
for(int i = 0; i < j; i++){ //loop A
student *ptr = new student(namesList[i], ageList[i]);   //creates the heap and adds the data in it (it triggers constructor)
studentList[i] = *ptr;                                  //assigns the stack and triggers the copy assignment to copy the data to the stack
                                                        //!note: when we assigned studentlist[i] = *ptr it copies the data not TRANSFER it
}
```

OUTPUT:

```
the constructor is called
the constructor is called
the constructor is called
the constructor is called
the constructor is called
the constructor is called
copy assignment operator is called
the constructor is called
copy assignment operator is called
the constructor is called
copy assignment operator is called
the constructor is called
copy assignment operator is called
the constructor is called
copy assignment operator is called
```

Observations:

I noticed that loop A's purpose is to create a heap and transfer the data (Name and age) to the stack and heap. First each student variable was created 5 times(it used john doe and, 18 as a place holder before the data is placed) hence why the constructor was called 5 times next it transferred the data in it and gets

added into the student class which is then copied by the constructor and placed it into the heap, then the copy assignment function is called which then creates a copy of the data(name, age) inside the heap and places it on the stack memory (which is the class “student”)

LOOP B:

```
for(int i = 0; i < j; i++){ //loop B
studentList[i].Print_Details(); //this loop is created to print the data of the studentList[i]
}
return 0;
}
```

OUTPUT:

```
Carly 15
Freddy 16
Sam 18
Zack 19
Cody 16
destructor is called
destructor is called
destructor is called
destructor is called
destructor is called
```

Observation:

once after it copies it, it then triggers loop B and prints the output, once it prints the output and all is finished it calls the destructor and deletes everything. The whole purpose of the loop B is just to print the output in the studentList[] array. (its basically a simple for loop output)

Another thing I noticed, normally when we use the “new” keyword we usually need to use “delete ptr” so that it deletes the data after using it but for this code we didn't use it so rather than the data deleting it just gets lost in the memory

Table 2-4. Modifications/Corrections Necessary

```
for(int i = 0; i < j; i++){ //loop A
student *ptr = new student(namesList[i], ageList[i]);   //creates the heap and adds the data in it (it triggers constructor)
studentList[i] = *ptr;                                  //assigns the stack and triggers the copy assignment to copy the data to the stack
                                                        //!note: when we assigned studentlist[i] = *ptr it copies the data not TRANSFER it
delete ptr;
}
```

Observation:

The only difference I added was a simple delete ptr in loop A because thats where it finishes the transferring/copying of data to the heap and stack. Once it finishes it, it then deletes the data it stored and thus fixing the “lost” data problem

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

```
class fruit : public Item {
public:
    // Constructor
    fruit(std::string newname = "Fruit", int newprice = 0, int newquantity = 0)
        : Item(newname, newprice, newquantity, "Fruit") {
        std::cout << "Fruit constructor called" << std::endl;
    }

    // Destructor
    ~fruit() {
        std::cout << "Fruit destructor called" << std::endl;
    }

    // Copy Constructor
    fruit(const fruit &copy) : Item(copy) {
        std::cout << "Fruit copy constructor called" << std::endl;
    }

    // Copy Assignment Operator
    fruit& operator=(const fruit &copy) {
        std::cout << "Fruit copy assignment called" << std::endl;
        if (this != &copy) {
            Item::operator=(copy);
        }
        return *this;
    }
};
```

Figure 3.1.1

Here is the class for fruit, from the procedure output I used it as a reference to create the copy assignment, constructor, destructor and copy assignment operator for the class fruit. But notice how it has a public variable called Item. which I will get to a bit but other than that the, fruit class itself has a similar structure to the student class back in the procedure part

```cpp
class vegetable : public Item {
public:
    // Constructor
    vegetable(std::string newname = "Vegetable", int newprice = 0, int newquantity = 0)
        : Item(newname, newprice, newquantity, "Vegetable") {
        std::cout << "Vegetable constructor called" << std::endl;
    }

    // Destructor
    ~vegetable() {
        std::cout << "Vegetable destructor called" << std::endl;
    }

    // Copy Constructor
    vegetable(const vegetable &copy) : Item(copy) {
        std::cout << "Vegetable copy constructor called" << std::endl;
    }

    // Copy Assignment Operator
    vegetable& operator=(const vegetable &copy) {
        std::cout << "Vegetable copy assignment called" << std::endl;
        if (this != &copy) {
            Item::operator=(copy);
        }
        return *this;
    }
};
```

Figure 3.1.2

The same applies to the vegetable class

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

```cpp
//MAIN CALLERR
int main() {
    //assigns the GroceryList array a size
    const int SIZE = 4;
    Item* GroceryList[SIZE];

    // PROBLEM 2: Dynamic allocation into GroceryList
    //assigns the array with the category, quantity and price
    GroceryList[0] = new fruit("Apple", 10, 7);
    GroceryList[1] = new fruit("Banana", 10, 8);
    GroceryList[2] = new vegetable("Broccoli", 60, 12);
    GroceryList[3] = new vegetable("Lettuce", 50, 10);

    //prints out the values in the array
    std::cout << "\n--- Displaying Items ---" << std::endl;
    for (int i = 0; i < SIZE; i++) {
        if (GroceryList[i] != NULL) {
            GroceryList[i]->printDetails();
        }
    }
```

Figure 3.2.1

In this section this is where the array GroceryList is created first I assigned with the size of the array then added in the values it has. But notice I use “new” here this will call out the fruit and vegetable functions consecutively and calls out both of their constructor function and it will store the memory in the heap and stack of the classes. Then after that it prints out the values of the array

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna’s Grocery List.

```
// PROBLEM 3: TotalSum Function

// the function to calculate the grand total of the items in jenna's list
int TotalSum(Item* GroceryList[], int size) {
    int grandTotal = 0;
    for (int i = 0; i < size; i++) {
        if (GroceryList[i] != NULL) {
            grandTotal += GroceryList[i]->calculateSum();
        }
    }
    return grandTotal;
}
```

```
//the function to calculate the sum
int calculateSum() {
    return price * quantity;
}
```

Figure 3.3.1

Here is the function to calculate for the total sum of the variables in the list. It checks the array for the contents in it specifically the price it then calls out the calculate sum function which is a simple function that multiplies the price with the quantity and outputs it

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

```
    // PROBLEM 4: Delete Lettuce and de-allocate memory

    //deletes letuce in the GroceryList

    for (int i = 0; i < SIZE; i++) {
        if (GroceryList[i] != NULL && GroceryList[i]->getName() == "Lettuce") {
            delete GroceryList[i];   // Free heap memory
            GroceryList[i] = NULL;   // Clear pointer to prevent dangling reference
            std::cout << "Lettuce deleted successfully." << std::endl;
        }
    }
    //outputs the grand total without the lettuce included
    std::cout << "Updated Grand Total: PHP " << TotalSum(GroceryList, SIZE) << std::endl;

    // Clean up remaining allocated memory
    for (int i = 0; i < SIZE; i++) {
        if (GroceryList[i] != NULL) {
            delete GroceryList[i];
            GroceryList[i] = NULL;
        }
    }

    return 0;
}
```

Figure 3.4.1

This section here finds the lettuce in jenna's list/array which then deletes it as well as deleting its contents in the heap memory and vegetable class. This code also outputs the total sum without the lettuce in it which uses the updated array list and computes the total sum using the function. Once its all finished it then cleans up the allocated memory inside it and finishes the program

EXTRA CODE

```cpp
#include <iostream>
#include <string>

// creates a class for item. it serves as a array for the item. since an array only uses one data type this class's purpose is to have-
// - multiple data types store in that array
class Item {
private: //stores the name,price, quantity and category in the item class
    std::string name;
    int price;
    int quantity;
    std::string category;

public:
    Item(std::string name = "Item", int price = 0, int quantity = 0, std::string category = "Item") {
        //stores it into "this" which is a built in pointer which stores the memory address of the price,name,quantity and category
        this->name = name;
        this->price = price;
        this->quantity = quantity;
        this->category = category;
    }

    ~Item() {}
     //this is a  Getter functions to retrieve the Item's member variables
     //NOTE! a getter is used to return the value of a private variable
    std::string getName() { return name; }
    int getPrice() { return price; }
    int getQuantity() { return quantity; }
    std::string getCategory() { return category; }
```

Figure 3.5.1

In this section back in the first problem (the vegetable and fruit class) the class calls a variable called “Item” this variable is actually a class made to help with the problem that I faced in problem2 which is the array only accepting one data type, this class helps fixes the array problem

```cpp
    //the function to calculate the sum
    int calculateSum() {
        return price * quantity;
    }
   //the function to print the shopping list of jenna
    void printDetails() {
        std::cout << "Category: [" << category << "] | Name: " << name
                  << " | Price: PHP " << price
                  << " | Qty: " << quantity
                  << " | Subtotal: PHP " << calculateSum() << std::endl;
    }
};
```

Figure 3.5.2

Here is where the as previously mentioned calculateSum function asides which is in the item class as well as a void function thats basically a label for the category, price, quantity and subtotal

COMPUTER ENGINEERING

QUEZON CITY

OUTPUT

```
Fruit constructor called
Fruit constructor called
Vegetable constructor called
Vegetable constructor called

--- Displaying Items ---
Category: [Fruit] | Name: Apple | Price: PHP 10 | Qty: 7 | Subtotal: PHP 70
Category: [Fruit] | Name: Banana | Price: PHP 10 | Qty: 8 | Subtotal: PHP 80
Category: [Vegetable] | Name: Broccoli | Price: PHP 60 | Qty: 12 | Subtotal: PHP 720
Category: [Vegetable] | Name: Lettuce | Price: PHP 50 | Qty: 10 | Subtotal: PHP 500
Grand Total: PHP 1370
Lettuce deleted successfully.
Updated Grand Total: PHP 870
PS C:\Users\PC\Documents\DSA CAMPO>
```

REFLECTION

For me personally this activity is a HUGE challenge for me because not only im limited in time but the material im given, I have no prior knowledge of, sure functions and loops were introduced back then but going back to c++ and relearning the fundamentals aswell as learning new thing is a huge challenge for me. I can say though that this activity helped me understand more about constructors, pointers, classes, copy constructors, copy assignment operators and destructors more. I just hope I can implement my learnings in future programming and I just hope I learn more new things here rather than focus on finishing this task.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explanation of concepts, idea generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ai to explain pretty much everything in this work, I asked to give me example to better visualize it and I asked questions about the work if I had any confusion**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

COMPUTER ENGINEERING

QUEZON CITY

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.